LECTURE 6: SEARCH ALGORITHMS

DS 8001

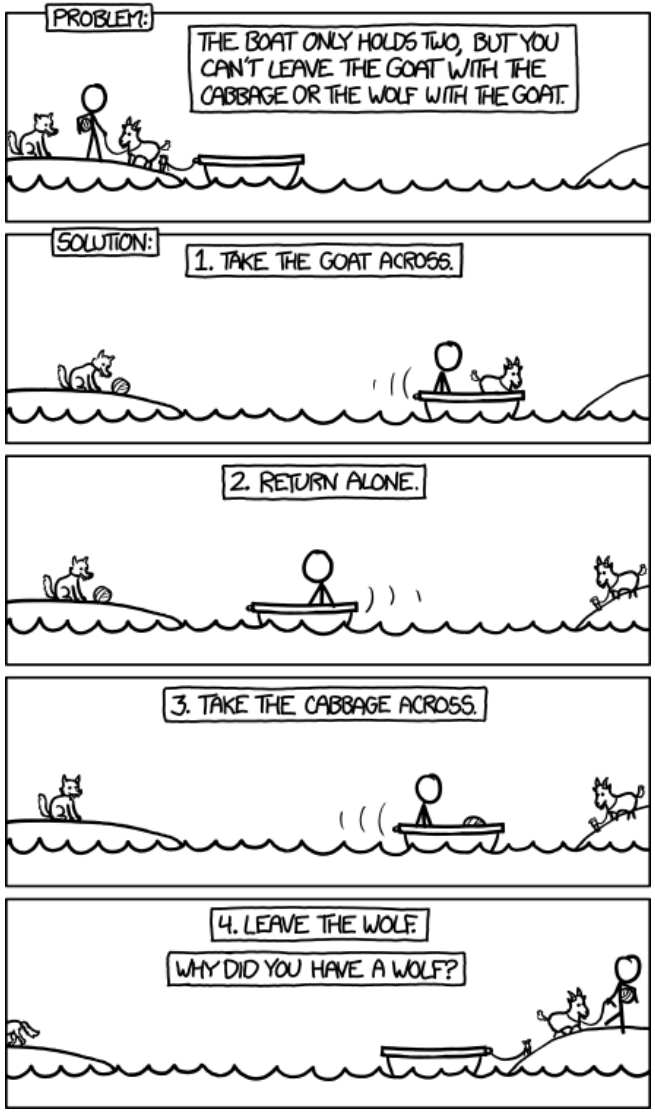
Table of Contents

1. Uninformed Search
   - Breadth-first search
   - Uniform-cost search
   - Depth-first search
   - Iterative deepening
   - Performance comparison and examples
2. Informed Search
3. Local Search for Optimization
   - Hill climbing
   - Simulated Annealing
   - Genetic Algorithm

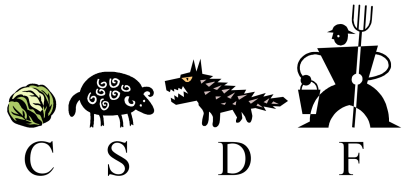
Toronto Metropolitan University

Search in AI

- Many AI problems can be formulated as search.
- Running example – river crossing: a farmer **F** must ferry a cabbage **C**, a sheep **S** and a dog **D** to the far bank. The boat holds F plus at most one of them, and if F is absent the dog eats the sheep and the sheep eats the cabbage.
- https://xkcd.com/1134/ (xkcd 1134, "Logic Boat": the same puzzle with a wolf and a goat)



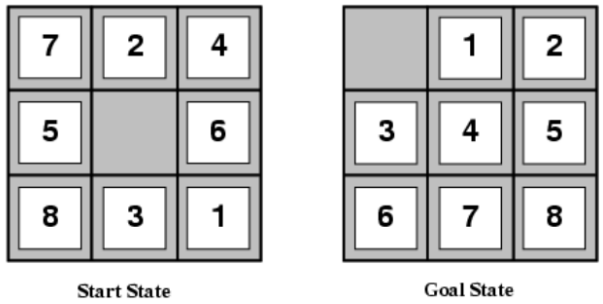
THE SEARCH PROBLEM



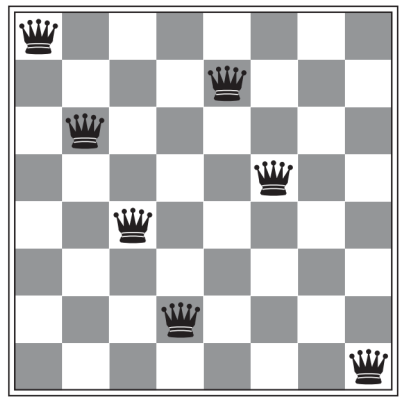
- Notation: write a state as (left bank, right bank); F also marks the side the boat is on.
- State space $\mathcal{S}$: all valid configurations (no bank holds C,S or S,D without F)
- Initial states (nodes) $\mathcal{I} = \{(CSDF,)\} \subseteq \mathcal{S}$
  - everything, boat included, starts on the left
- Goal states $\mathcal{G} = \{(, CSDF)\} \subseteq \mathcal{S}$ – the left bank is empty
- Successor function $succs(s) \subseteq \mathcal{S}$: states reachable in one step (one arc) from s
  - $succs((CSDF,)) = \{(CD, SF)\}$ – S must go first
  - $succs((CDF, S)) = \{(CD, SF), (D, CFS), (C, DSF)\}$
- $\text{Cost}(s, s') = 1$ for all arcs
- The search problem: find a solution path from a state in $\mathcal{I}$ to a state in $\mathcal{G}$
  - Optionally minimize the cost of the solution

SEARCH EXAMPLES

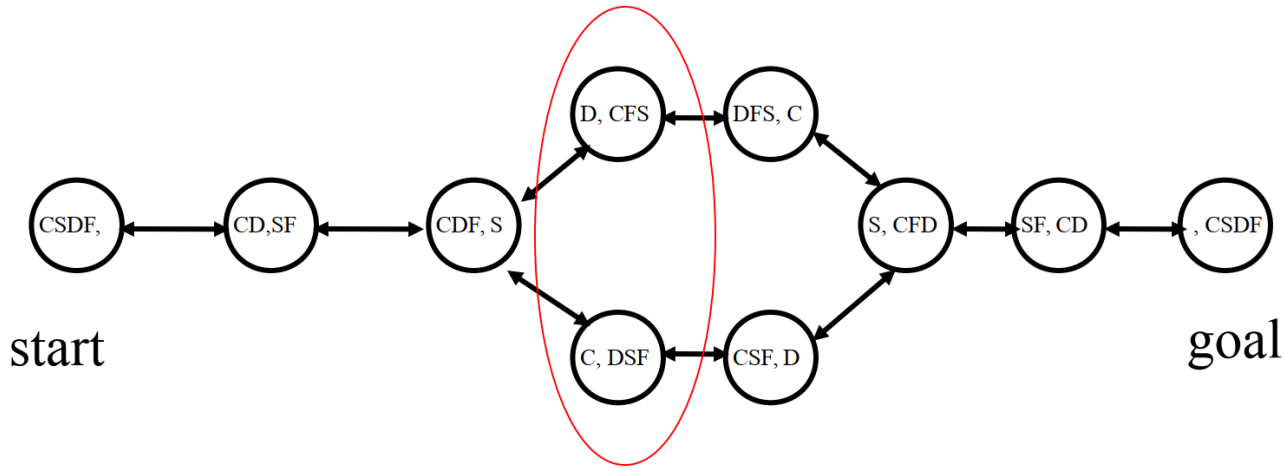
- 8-puzzle
  - States = configurations
  - successor function = up to 4 kinds of movement (slide the blank)
  - Cost = 1 for each move



- 8-queens (incremental formulation; Section 3 uses one queen per column instead)
  - States: Any arrangement of 0 to 8 queens on the board is a state
  - Actions: Add a queen to any empty square
  - Transition model: Returns the board with a queen added to the specified square
  - Goal test: 8 queens are on the board, none attacked.



A DIRECTED GRAPH IN STATE SPACE



Once S is across, F takes either C or D (circled); both branches give a shortest solution of 7 crossings.

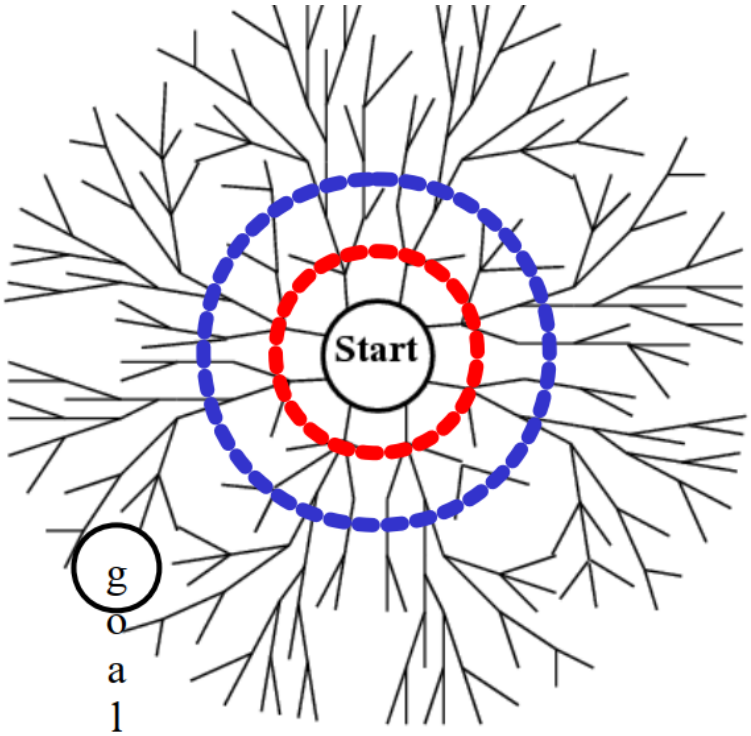
- To **expand** a state is to apply *succs*() to it; the states returned are **generated**.
- In general there will be many generated, but unexpanded states at any given time
- The generated, but not yet expanded states form the fringe (OPEN)
- One has to choose which one to expand next
  - Deep or shallow?

UNINFORMED SEARCH ON TREES

- Uninformed means we only know:
  - The goal test
  - The succs() function
- But not which non-goal states are better: that would be *informed* search
- Assume succs() graph is a tree
  - Won't encounter repeated states
  - This assumption can be relaxed (see "If state space graph is not a tree")
- Search strategies: breadth-first (BFS), uniform-cost (UCS), depth-first (DFS), iterative deepening (IDS), bidirectional (BIBFS)
- Differ in which unexpanded node to expand next – i.e. by the order imposed on the fringe

BREADTH-FIRST SEARCH (BFS)

- Expand the shallowest node first
  - Examine states one step away from the initial states
  - Examine states two steps away from the initial states
  - and so on...

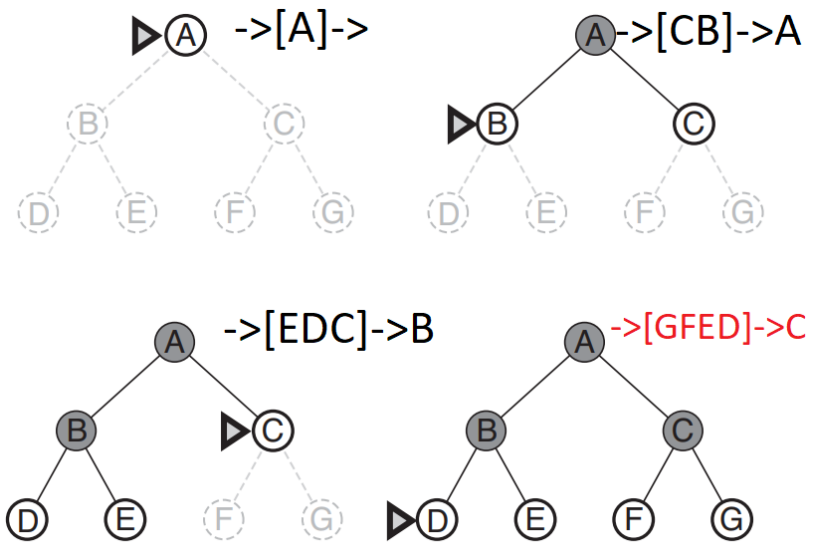


The explored region grows as a *ripple* of increasing radius around the start.

BFS Algorithm

Use a queue (First-in First-out, FIFO)

```
en_queue(initial states)
while (queue not empty) do
    s = de_queue()
    if (s==goal) then
        success!
    end if
    T = succs(s)
    for t in T: do
        # back ptr to recover soln
        t.prev = s
    end for
    en_queue(T)
end while
return failure
```



- The bracket is the queue: states enter on the left, leave on the right.
- If G is goal, we have seen it, but we do not stop
  - ->[GFED]->C
  - ->[GFE]->D
  - ...
- The goal test fires on *de-queue*, not on generation, so G waits behind D, E, F.

Performance of BFS

Assume the graph may be infinite but a goal exists finitely many steps away. b = *branching factor*, d = depth of the shallowest goal; time counts states generated, space counts states stored.
Four measures of search algorithms:

- **Completeness** (is *a* goal found whenever one exists?): yes, BFS will find a goal
- **Optimality** (is that goal a cheapest one?): yes if edges cost 1 (more generally, if path cost is a non-decreasing function of depth), no otherwise (solution: Uniform-cost search)
- **Time complexity** (worst case): goal is the last node at radius d
  - Have to generate all nodes at radius d
  - $b + b^2 + \ldots + b^d \sim O(b^d)$
  - Goal test on de-queue (as in the algorithm): one more level, $O(b^{d+1})$ time and space
- **Space complexity**: bad (solution: DFS)
  - Back pointers for all generated nodes $O(b^d)$
  - The queue / fringe (smaller, but still $O(b^d)$)

Toronto Metropolitan University

UNIFORM-COST SEARCH

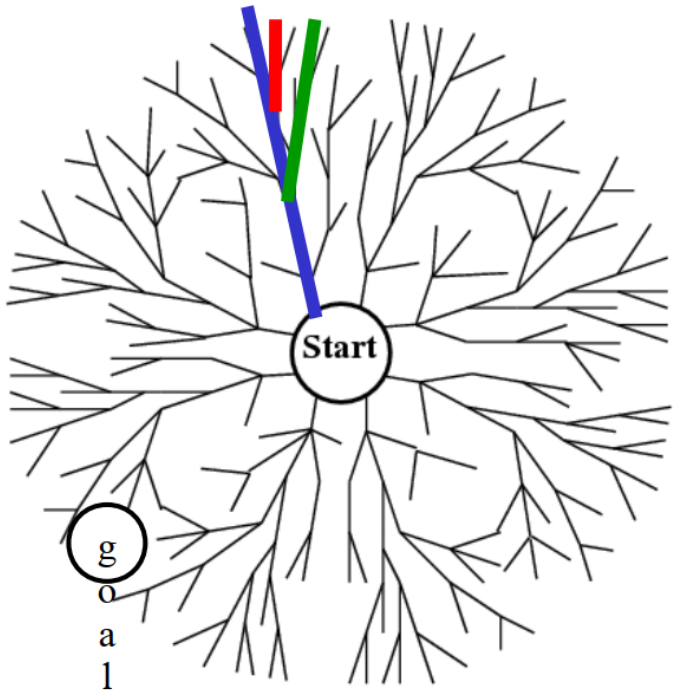
- Find the least-cost goal
- Each node has a path cost from start (= sum of edge costs along the path), written $g(s)$. Expand the least cost node first
- Use a priority queue instead of a normal queue
  - Always take out the least cost item
  - Goal test when a node is taken out, not when it is generated: the first path generated to a goal need not be the cheapest
- Complete and optimal (if edge costs $\geq \epsilon > 0$)
- Time and space: can be much worse than BFS
  - Let C^* be the cost of the least-cost goal
  - $O(b^{C^*/\epsilon})$, possibly $C^*/\epsilon \gg d$
  - With unit costs $\epsilon = 1$ and $C^* = d$, so UCS reduces to BFS

Toronto Metropolitan University

DEPTH-FIRST SEARCH (DFS)

- Expand the deepest node first
  1. Select a direction, go deep to the end
  2. Slightly change the end
  3. Slightly change the end some more

     fan...



The coloured strokes are the first three root-to-leaf paths: a *fan*, not a ripple.

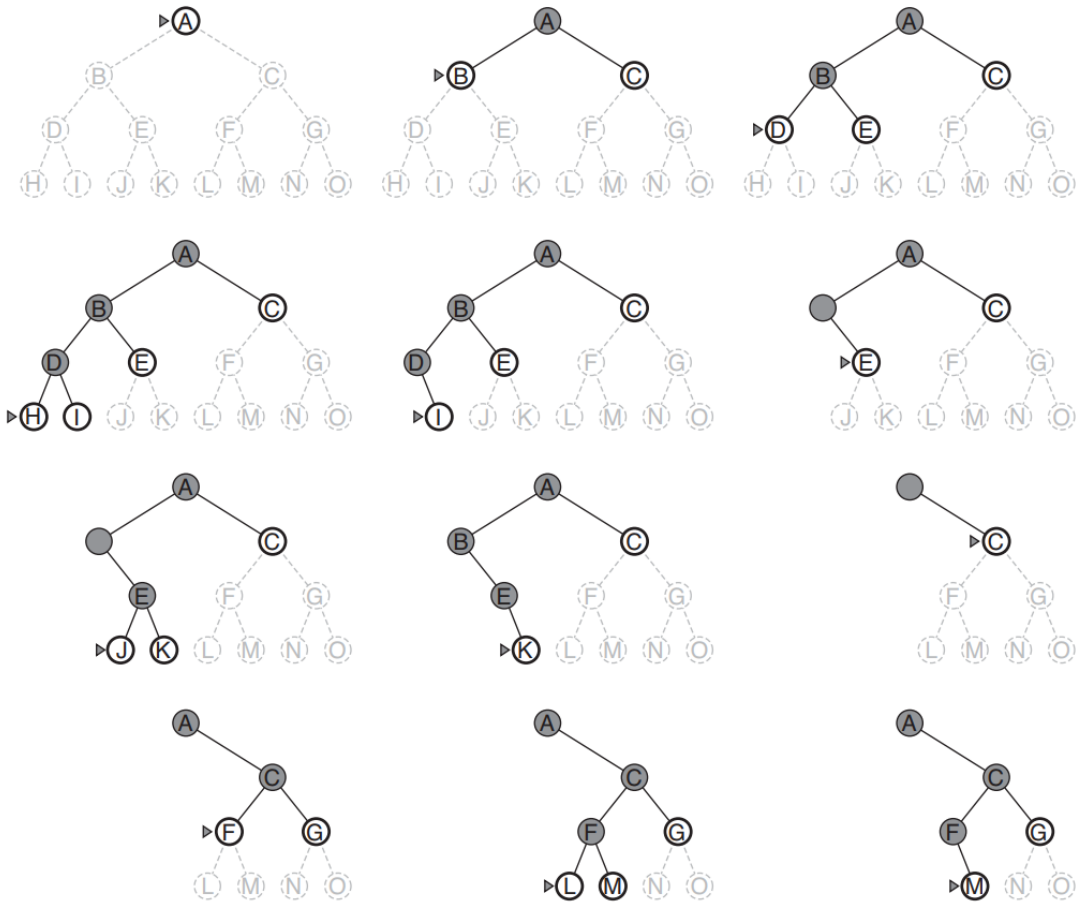
DFS Algorithm

Use a stack (last-in, first-out, LIFO)

```
push(initial states)
while (stack not empty) do
    s = pop()
    if (s==goal) then
        success!
    end if
    T = succs(s)
    for t in T: do
        t.prev = s
    end for
    push(T)
end while
return failure
```

Identical to BFS except for the container: a stack, not a queue.

DFS Example



Read left to right, top to bottom. Shaded = expanded, dashed = not yet generated, triangle = about to be expanded. A shaded node that loses its label has a finished subtree and is discarded: only one root-to-leaf path and its siblings are ever in memory.

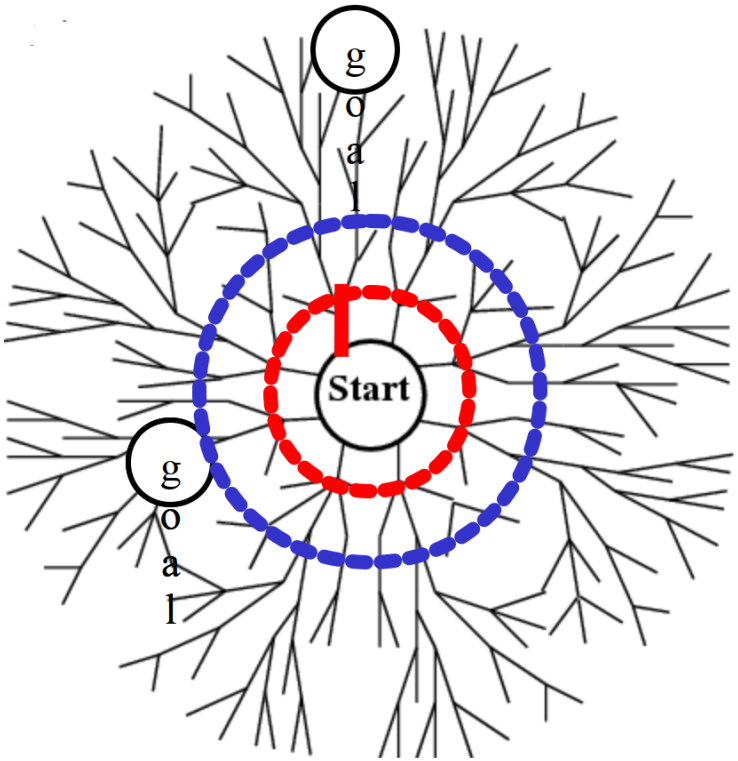
PERFORMANCE OF DFS

- m = maximum depth of graph from start
- Space complexity: $m(b-1) \sim O(mb)$
  - "backtracking search" even less space – generate siblings (if applicable) one at a time, giving $O(m)$
- Time complexity: may visit almost all nodes $\rightarrow O(b^m)$ (for finite trees)
- May not be optimal – it returns the first goal reached, not the shallowest or cheapest
- Not complete either when the space is infinite or cyclic: DFS can descend one branch forever. On a finite tree it is complete.

How about combining BFS and DFS?

- DFS but stop if path length > 1 (*depth-limited* DFS)
- If goal not found, repeat DFS, stop if path length > 2
- And so on ...

  fan within ripple



Each depth limit sweeps a DFS fan inside a BFS ripple of that radius.

ITERATIVE DEEPENING

- Search proceeds like BFS, but fringe is like DFS
  - Complete, optimal like BFS
  - Small space complexity like DFS – $O(bd)$, not $O(b^d)$
- Huge waste?
  - Each deepening repeats DFS from the beginning
    $$db + (d-1)b^2 + (d-2)b^3 + \ldots + b^d \sim O(b^d)$$
  - Depth-k nodes are regenerated $d-k+1$ times, but the deepest level dominates, so for $b \geq 2$ this costs only a constant factor
  - Time complexity like BFS
- Preferred uninformed search method when the goal depth d is not known in advance

PERFORMANCE OF SEARCH ALGORITHMS ON TREES

b: branching factor (assume finite) d: goal depth m: graph depth

	Complete	optimal	time	space
Breadth-first search	Y	Y, if [1]	$O(b^d)$	$O(b^d)$
Uniform-cost search[2]	Y	Y	$O(b^{C^*/\varepsilon})$	$O(b^{C^*/\varepsilon})$
Depth-first search	N	N	$O(b^m)$	$O(bm)$
Iterative deepening	Y	Y, if [1]	$O(b^d)$	$O(bd)$

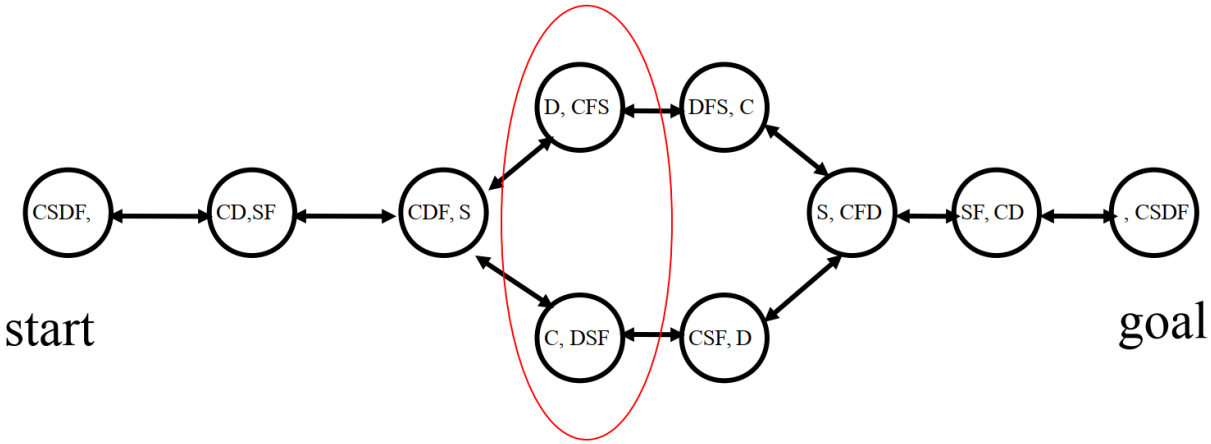
1. edge cost constant, or positive non-decreasing in depth
• edge costs $\geq \varepsilon > 0$. C* is the best goal path cost.

The "N" for DFS refers to infinite or cyclic spaces (on a finite tree DFS is complete).

Bidirectional search (BFS from both start and goal) can be used to reduce time and space complexity to $O(b^{d/2})$ — may not always be feasible! It needs an explicit goal state and a backward successor function.

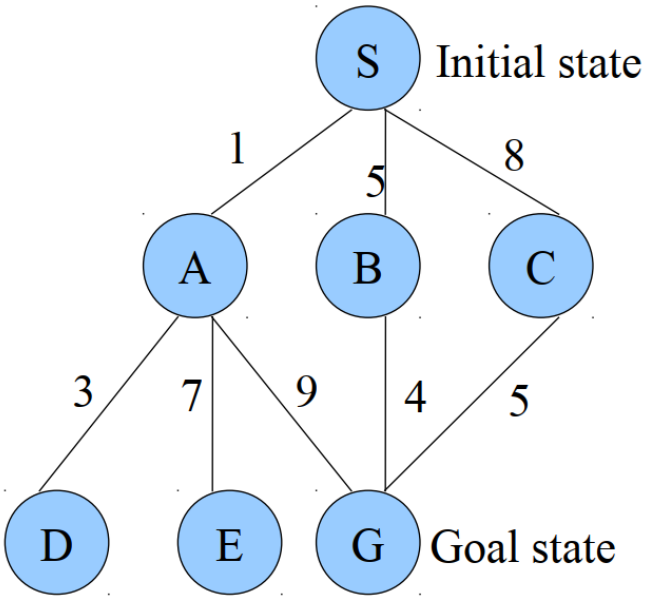
IF STATE SPACE GRAPH IS NOT A TREE

The problem: repeated states



- Ignore danger of repeated states: wasteful (BFS) or non-terminating (DFS).
- Solution idea (*graph search*, as opposed to *tree search*):
  - We have to remember already-expanded states (CLOSED)
  - When we take out a state from the fringe (OPEN), check whether it is in CLOSED (already expanded)
    - ✓ If yes, throw it away
    - ✓ If no, expand it (add successors to OPEN), and move it to CLOSED
  - Price: CLOSED grows with the number of distinct states

WORKED EXAMPLE: THE FOUR STRATEGIES COMPARED



All edges are directed, pointing downwards

Nodes expanded by

- Depth-First Search: S A D E G
  Solution found: S A G
- Breadth-First Search: S A B C D E G
  Solution found: S A G
- Uniform-cost Search: S A D B C E G
  Solution found: S B G (This is the only uninformed search that worries about costs.) Cost 9, against 10 for S A G.
- Iterative-Deepening Search: S A B C S A D E G
  Solution found: S A G (depth limits 1 and 2; the limit-0 iteration expands only S)

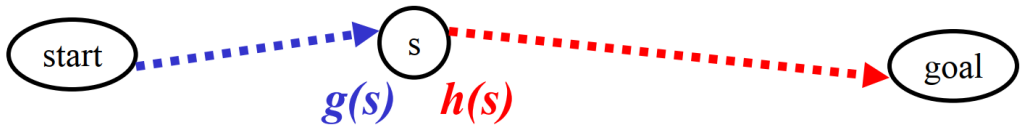
Children are generated left to right, so DFS tries A before B and C. UCS takes out C before E (both cost 8) because C was queued first.

UNINFORMED VS. INFORMED SEARCH

- **Uninformed search** (BFS, uniform-cost, DFS, IDS etc.)
  - Knows the actual path cost g(s) from start to a node s in the fringe, but that is it.



- **Informed search**
  - also has a heuristic h(s) of the cost from s to goal. ('h'= heuristic, non-negative)

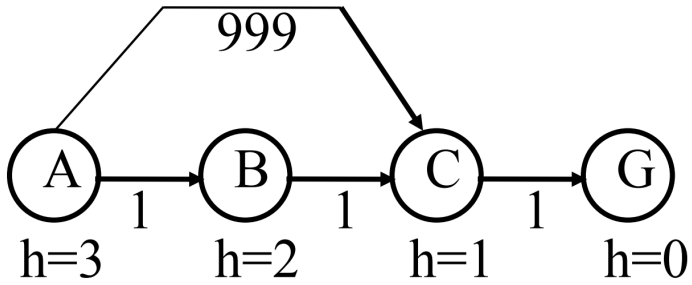


  - Can be much faster than uninformed search.

- Write $h^*(s)$ for the *true* cheapest cost from s to a goal; $h(s)$ estimates $h^*(s)$ from the state alone, without searching. UCS already orders its queue by $g(s)$ – how should $h(s)$ be used too?

First attempt: Greedy best-first search

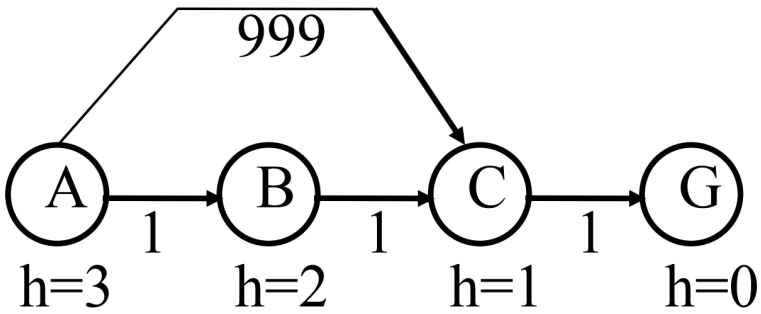
- Idea 1: use h(s) instead of g(s)
- Always expand the node with the least h(s)
- Use a priority queue:
  - Push in states with their estimated cost-to-go h(s)
  - Pop out the state with the least h(s) first.
- Known as "greedy best-first" search
- How is this idea?



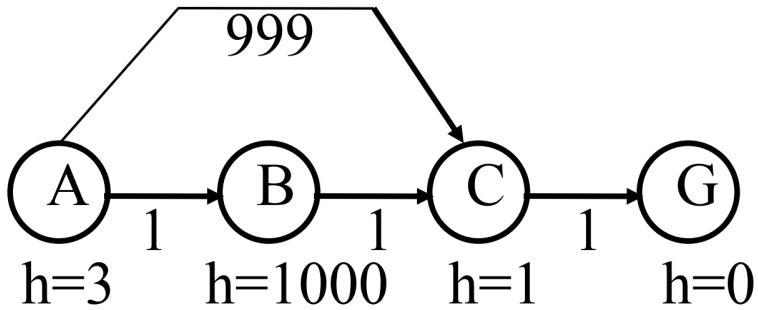
  - It will follow path $A \rightarrow C \rightarrow G$ – it prefers C (h=1) to B (h=2)
  - Obviously not optimal – cost 1000 instead of 3, because g is ignored

SECOND ATTEMPT: A SEARCH

- Idea 2: use g(s) + h(s), written $f(s)$
- Always expand the node with the least g(s) + h(s)
- Use a priority queue:
  - Push in states with their priority g(s) + h(s)
  - Pop out the state with the least g(s) + h(s) first.
- Known as "A" search
- How is this idea?



- Works for this example!
  $f(B) = 3 < f(C) = 1000$



- A search is not optimal –
  $h(B) = 1000 \gg h^*(B) = 2$, so
  $f(B) > f(C)$

THIRD ATTEMPT: A* SEARCH

- Same as A search, but the heuristic function $h()$ has to satisfy $0 \leq h(s) \leq h^*(s)$
  - Such heuristic function $h()$ is called admissible
  - An admissible heuristic never over-estimates

    $\Rightarrow$ It is always optimistic
  - The bad example above is now excluded: $h(B) = 1000 > h^*(B) = 2$
- A search with admissible $h()$ is called A* search

ADMISSIBLE HEURISTIC FUNCTIONS h - 1

- 8-puzzle example (here the goal puts the blank at the lower right)

Example State

1		5
2	6	3
7	4	8

Goal State

1	2	3
4	5	6
7	8	

- Which of the following are admissible heuristics?
  - $h(n)$=number of tiles in wrong position

    $\Rightarrow$ Yes
  - $h(n) = 0$

    $\Rightarrow$ Yes, A* then reduces to uniform-cost search
  - $h(n) = 1$

    $\Rightarrow$ No, at a goal $h^* = 0$ but $h = 1$: an over-estimate
  - $h(n) =$ sum of Manhattan distance between each tile and its goal location

    $\Rightarrow$ Yes
- Check on the example state: 6 tiles are misplaced, and the Manhattan distances sum to 9.

Toronto Metropolitan University

ADMISSIBLE HEURISTIC FUNCTIONS h - 2

- In general, which of the following are admissible heuristics? $h^*(n)$ is the true optimal cost from n to goal
  - $h(n) = h^*(n)$

    $\Rightarrow$ Yes – the perfect heuristic
  - $h(n) = \max\{2, h^*(n)\}$

    $\Rightarrow$ No, over-estimates when $h^*(n) < 2$
  - $h(n) = \min\{2, h^*(n)\}$

    $\Rightarrow$ Yes – admissible but very weak
  - $h(n) = h^*(n) - 2$

    $\Rightarrow$ No, possibly negative (at a goal it gives -2, although it never over-estimates)
  - $h(n) = \sqrt{h^*(n)}$

    $\Rightarrow$ No, if $0 < h^*(n) < 1$

DESIGNING ADMISSIBLE HEURISTICS

- How to construct heuristic functions?



- Often by relaxing the constraints – the exact cost of a relaxed problem is admissible for the original
  - $h_1(n)$=number of tiles in wrong position $\Rightarrow$ Allow tiles to fly to their destination in one step
  - $h_2(n)$=sum of Manhattan distance between each tile and its goal location $\Rightarrow$ Allow tiles to move on top of other tiles
- A heuristic function h_2 **dominates** h_1 if for all s: $h_1(s) \leq h_2(s) \leq h^*(s)$ – e.g. $6 \leq 9$ for the state above
- We prefer heuristic functions as close to h^* as possible, but not over h^*. A dominating heuristic never expands more nodes.
- But... Good heuristic function might need complex computation

  $\Rightarrow$ Time may be better spent, if we use a faster, simpler heuristic function and expand more nodes

Toronto Metropolitan University

THE A* ALGORITHM

OPEN is the priority queue of generated-but-unexpanded nodes, CLOSED the expanded ones; the priority is $f(n) = g(n) + h(n)$, and $c(n, n')$ is the cost of arc $n \to n'$.

1. Put the start node S on the priority queue, called OPEN
2. If OPEN is empty, exit with failure
3. Remove from OPEN and place on CLOSED a node n for which f(n) is minimum
4. If n is a goal node, exit (trace back pointers from n to S)
5. Expand n, generating all its successors and attach to them pointers back to n. For each successor n' of n
    1. If n' is not already on OPEN or CLOSED estimate h(n'),g(n')=g(n)+ c(n,n'), f(n')=g(n')+h(n'), and place it on OPEN.
    2. If n' is already on OPEN or CLOSED, then check if g(n') is lower for the new version of n'. If so, then:
        1. Redirect pointers backward from n' along path yielding lower g(n').
        2. Put n' on OPEN.
    3. If g(n') is not lower for the new version, do nothing.
6. Goto 2.

Step 5.2 returns a node from CLOSED to OPEN when a cheaper path appears; that is what keeps A* optimal on graphs for any admissible h. If h is also *consistent* ($h(n) \le c(n, n') + h(n')$ on every arc), a CLOSED node never gets a cheaper path, so step 5.2 never re-opens one.

THE A* ALGORITHM - OBSERVATIONS

- A* can use a lot of memory

  $\Rightarrow$ O(number of states)

- For large problems, A* will run out of memory
- Two alternatives might be considered:
  - Iterative deepening A* (IDA*) – repeated depth-first searches cut off by a bound on f instead of on depth; memory linear in depth
  - Beam search – keep only the k best fringe nodes and discard the rest; memory $O(k)$, but completeness and optimality are lost

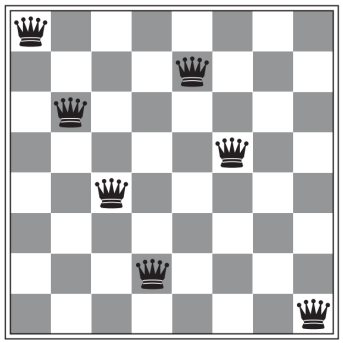
Optimization problems

- Previously we wanted a path from start to goal
  - Uninformed search: g(s)– uniform-cost search
  - Informed search: g(s)+h(s)– A*
- Now a different setting:
  - Each state s has a score $f(s)$ that we can compute (the *objective* or *fitness* function)
  - The goal is to find the state with the highest score, or a reasonably high score
  - Do not care about the path
  - This is an optimization problem
  - Enumerating the states is intractable
  - Even previous search algorithms are too expensive

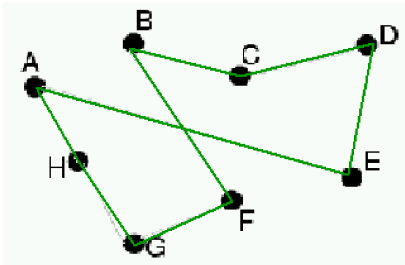
Toronto Metropolitan University

Optimization examples

- N-queen
  - $f(s)$= number of pairs of queens attacking each other in state s
  - Note we want s with the lowest score $f(s) = 0$. The techniques are the same. Low or high should be obvious from context.



- TSP (travelling salesperson problem)
  - Visit each city once, return to first city
  - State = order of cities, $f(s)$ = total mileage, minimized



- Boolean SAT (e.g. 3-SAT) – make a conjunction of clauses true; each 3-SAT clause has 3 literals
  - $f(s)$ = # of satisfied clauses
  - State = Assignment to variables

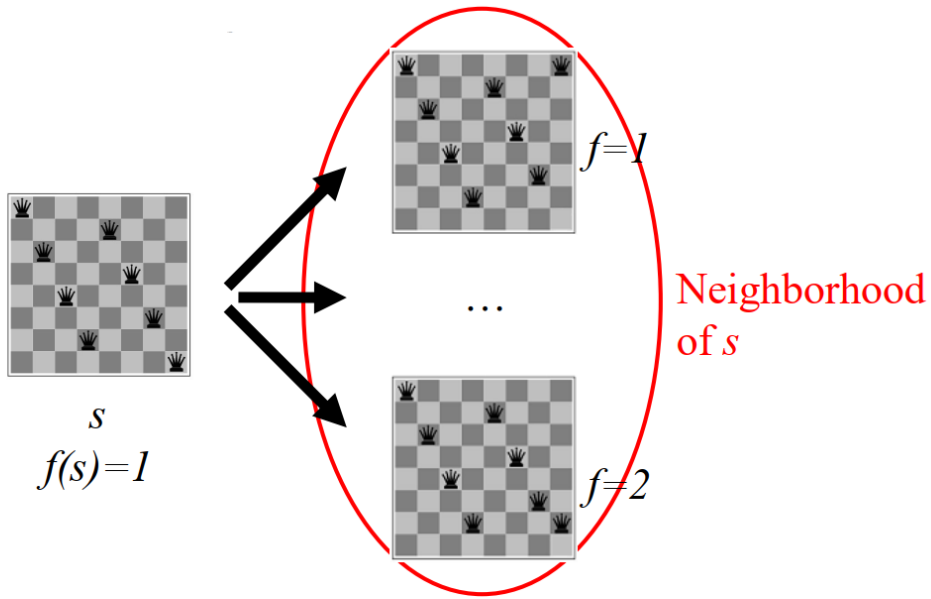
$A \vee \neg B \vee C$
$\neg A \vee C \vee D$
$B \vee D \vee \neg E$
$\neg C \vee \neg D \vee \neg E$
$\neg A \vee \neg C \vee E$

HILL CLIMBING

- Very simple idea: Start from some state s
  - Move to a neighbor t with better score. Repeat.
- Question: What is a neighbor?
  - You have to define that!
  - The neighborhood of a state is the set of neighbors
  - Also called 'move set'
  - Similar to the successor function, except that no path is stored

NEIGHBORS: N-QUEEN

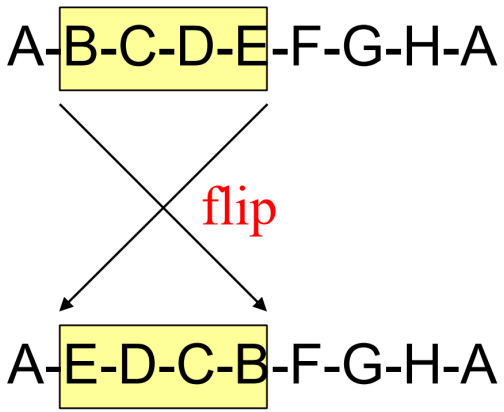
- Example: N-queen (one queen per column). One possibility:
  - Pick the most-conflicting column (the rightmost one, if tied)
  - Move the queen in that column vertically to a different location
- One queen per column removes all column conflicts by construction, so f only counts row and diagonal attacks.

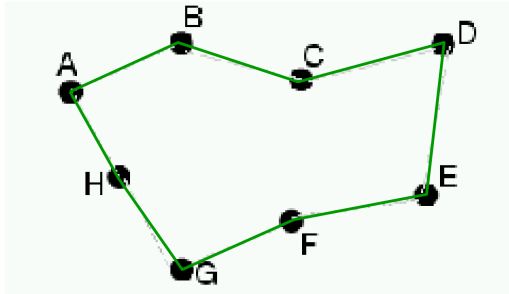


s has one attacking pair; some neighbors keep $f = 1$, others are worse ($f = 2$).

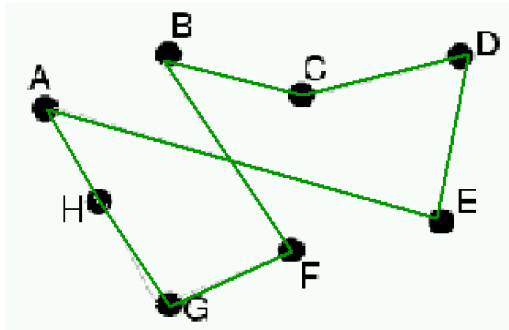
NEIGHBORS: TSP

- state = A-B-C-D-E-F-G-H-A
- f = length of tour
- One possibility: 2-change – reverse a block of the tour
- Reversing B..E swaps two edges: A–B and E–F become A–E and B–F





before: A-B-C-D-E-F-G-H-A



after: A-E-D-C-B-F-G-H-A – longer, so hill climbing rejects it; run the other way, 2-changes remove crossings

NEIGHBORS: SAT

- state = (A=T, B=F, C=T, D=T, E=T)
- f = # of satisfied clauses
- Neighbor: flip the assignment of one variable – 5 variables, so 5 neighbors (left), for the formula on the right
- Here $f(s) = 4$: the only unsatisfied clause is $\neg C \vee \neg D \vee \neg E$

(A=**F**, B=F, C=T, D=T, E=T)
(A=T, B=**T**, C=T, D=T, E=T)
(A=T, B=F, C=**F**, D=T, E=T)
(A=T, B=F, C=T, D=**F**, E=T)
(A=T, B=F, C=T, D=T, E=**F**)

$A \vee \neg B \vee C$
$\neg A \vee C \vee D$
$B \vee D \vee \neg E$
$\neg C \vee \neg D \vee \neg E$
$\neg A \vee \neg C \vee E$

HILL CLIMBING: DESIGN CHOICES

- **Question**: How should the neighborhood be designed?
  - (vaguely) Problems tend to have structures. A small change produces a neighboring state.
  - The neighborhood must be small enough for efficiency, yet connected enough to reach good states
  - Designing the neighborhood is critical. This is the real ingenuity - not the decision to use hill climbing.
- **Question**: Pick which neighbor?

  $\Rightarrow$ The best one (greedy)
- **Question**: What if no neighbor is better than the current state?

  $\Rightarrow$ Stop!

HILL CLIMBING ALGORITHM

1: Pick initial state s
2: Pick t in neighbors(s) with largest $f(t)$
3: **if** $f(t) \leq f(s)$ **then**
4: return s
5: STOP
6: **end if**
7: $s = t$. Go to Step 2.

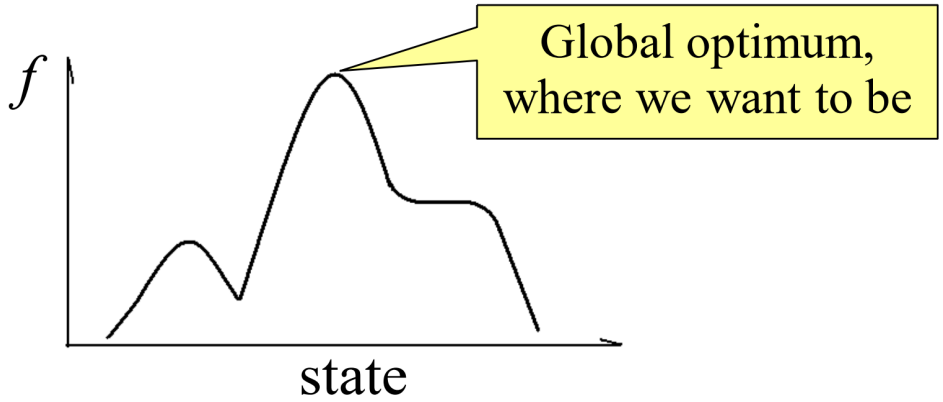
- Not the most sophisticated algorithm in the world.
- Very greedy
- Easily stuck

  $\Rightarrow$ Local optima!
- Written for maximization; reverse the comparison to minimize (e.g. queen conflicts).

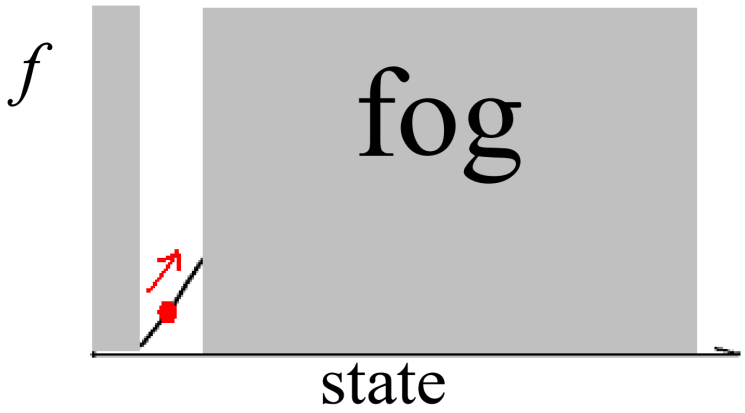
Toronto Metropolitan University

LOCAL OPTIMA IN HILL CLIMBING - 1

- Useful conceptual picture: f surface = 'hills' in state space



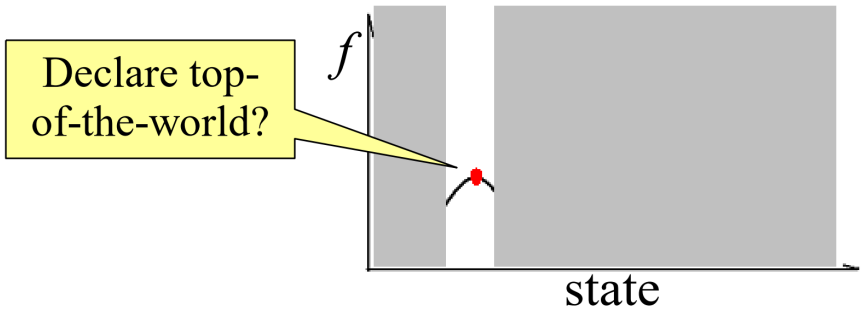
- But we can not see the landscape all at once. Only see the neighborhood. Climb in fog.



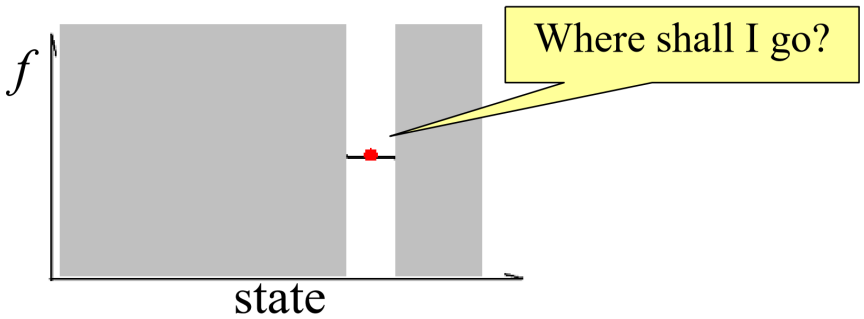
The horizontal axis is the state space; all the algorithm sees is f at s and at its neighbors.

LOCAL OPTIMA IN HILL CLIMBING - 2

- Local optima (there can be many!) – at least as good as every neighbor, yet worse than the global optimum; hill climbing stops and cannot tell



- Plateaux – flat regions where all neighbors score alike, so there is no gradient to follow



- How do we escape local optima?

REPEATED HILL CLIMBING WITH RANDOM RESTARTS

- Very simple modification

Repeated hill climbing:

1: When stuck, pick a random new start, run basic hill climbing from there
2: Repeat this k times.
3: Return the best of the k local optima

- Can be very effective
- If one run succeeds with probability p, about $1/p$ restarts are needed on average
- Should be tried whenever hill climbing is used

VARIATIONS OF HILL CLIMBING

- **Question**: How do we make hill climbing less greedy?
  - Stochastic hill climbing
    - ✓ Randomly select among better neighbors
    - ✓ The better, the more likely
    - ✓ Pros / cons compared with basic hill climbing? Slower, but may reach a better local optimum
- **Question**: What if the neighborhood is too large to enumerate? (e.g. N-queen if we need to pick both the column and the move within it)
  - First-choice hill climbing
    - ✓ Randomly generate neighbors, one at a time
    - ✓ If better, take the move
    - ✓ Pros / cons compared with basic hill climbing? Cheaper per step, but takes the first improvement, not the best
- We are still greedy! Only willing to move upwards.
  $\Rightarrow$ Sometimes one needs to temporarily step back to move forward.
  $\Rightarrow$ Sometimes one needs to move to an inferior neighbor in order to escape a local optimum.

WALKSAT

1: Pick a random unsatisfied clause
2: Consider 3 neighbors: flip each variable of that clause
3: **if** any improves f **then**
4: accept the best
5: **end if**
6: **if** none improves f **then**
7: 50% of the time pick the least bad neighbor
8: 50% of the time pick a random neighbor among the three
9: **end if**
10: Repeat until every clause is satisfied (or a flip budget runs out)

- One of the best-known local-search methods for SAT. Like all local search it is *incomplete*: it can find a satisfying assignment but never prove there is none.
- Example (SAT slide above): the only unsatisfied clause is $\neg C \vee \neg D \vee \neg E$; flipping C to F satisfies it and keeps the other four, so $f : 4 \rightarrow 5$ and the formula is solved.

Simulated Annealing (SA)

1: Pick initial state s
2: Randomly pick t in neighbors(s)
3: **if** $f(t)$ better than $f(s)$ **then**
4: always accept $s \leftarrow t$
5: **else**
6: accept $s \leftarrow t$ *with a small probability* p
7: **end if**
8: Go to 2 until bored!

- An inferior neighbor is sometimes accepted – that is how SA escapes local optima.
- How to choose the small probability?
  - idea 1: $p = 0.1$
  - idea 2: p decreases with time
  - idea 3: p decreases with time, also as the 'badness' $|f(s) - f(t)|$ increases

SA Solution Acceptance Probability

- If $f(t)$ better than $f(s)$, always accept t
- Otherwise, accept t with probability

$$\exp\left(-\frac{|f(s)-f(t)|}{Temp}\right) \qquad \rightarrow \text{Metropolis rule (Boltzmann factor)}$$

- Temp is a temperature parameter that 'cools' (anneals) over time, e.g. $Temp \leftarrow Temp * 0.9$, which gives $Temp_k = T_0\,(0.9)^k$ after k iterations
  - High temperature: almost always accept any t (random walk)
  - Low temperature: first-choice hill climbing (only improving moves survive)
- If the 'badness' (formally known as energy difference) $|f(s)-f(t)|$ is large, the probability is small.
- Concept check: for $|f(s)-f(t)| = 2$, acceptance is $e^{-2/4} \approx 0.61$ at $Temp = 4$ but only $e^{-4} \approx 0.018$ at $Temp = 0.5$.

Toronto Metropolitan University

SA Issues

- Cooling scheme important – too fast and SA freezes in a local optimum, too slow and it just wanders
- Neighborhood design is the real ingenuity, not the decision to use simulated annealing.
- Theory offers little practical guidance
  - With sufficiently slow cooling rate, finds global optimum with probability 1 (a schedule as slow as $Temp_k \propto 1/\log(1+k)$ suffices for a large enough constant, and is far too slow to use)
- The acceptance rule is due to Metropolis et al. (1953); simulated annealing as an optimization method was proposed by Kirkpatrick, Gelatt and Vecchi (1983), based on the analogy that alloys manage to find a near global minimum energy state, when annealed slowly
- Easy to implement
- Try hill-climbing with random restarts first!

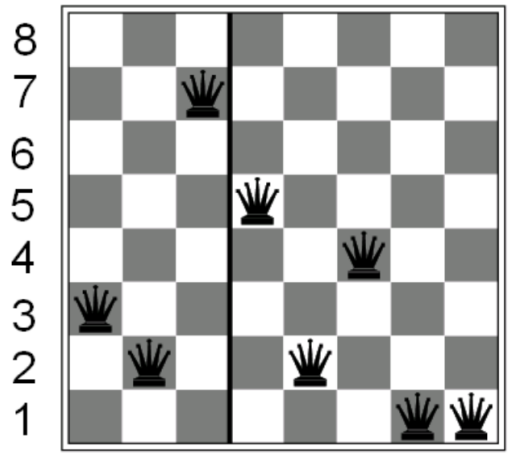
EVOLUTION AND NATURAL SELECTION

- Survival of the fittest, a.k.a. natural selection
  - Individuals with better genes have a larger chance to produce offspring, and vice versa
  - Superiority defined by fitness to the environment – there is no absolute scale
- The chromosomes from the parents exchange randomly by a process called **crossover**. Therefore, the offspring exhibit some traits of the father and some traits of the mother.
  - Requires genetic diversity among the parents to ensure sufficiently varied offspring
- A rarer process called **mutation** also changes the genes (e.g. from cosmic rays).
  - Nonsensical/deadly mutated organisms die.
  - Beneficial mutations produce "stronger" organisms
  - Neither: organisms are not improved

Toronto Metropolitan University

GENETIC ALGORITHM (GA) - 1

- Yet another AI algorithm based on a real-world analogy
- Yet another heuristic stochastic search algorithm
- Each state s is called an individual. Often (carefully) coded up as a string.



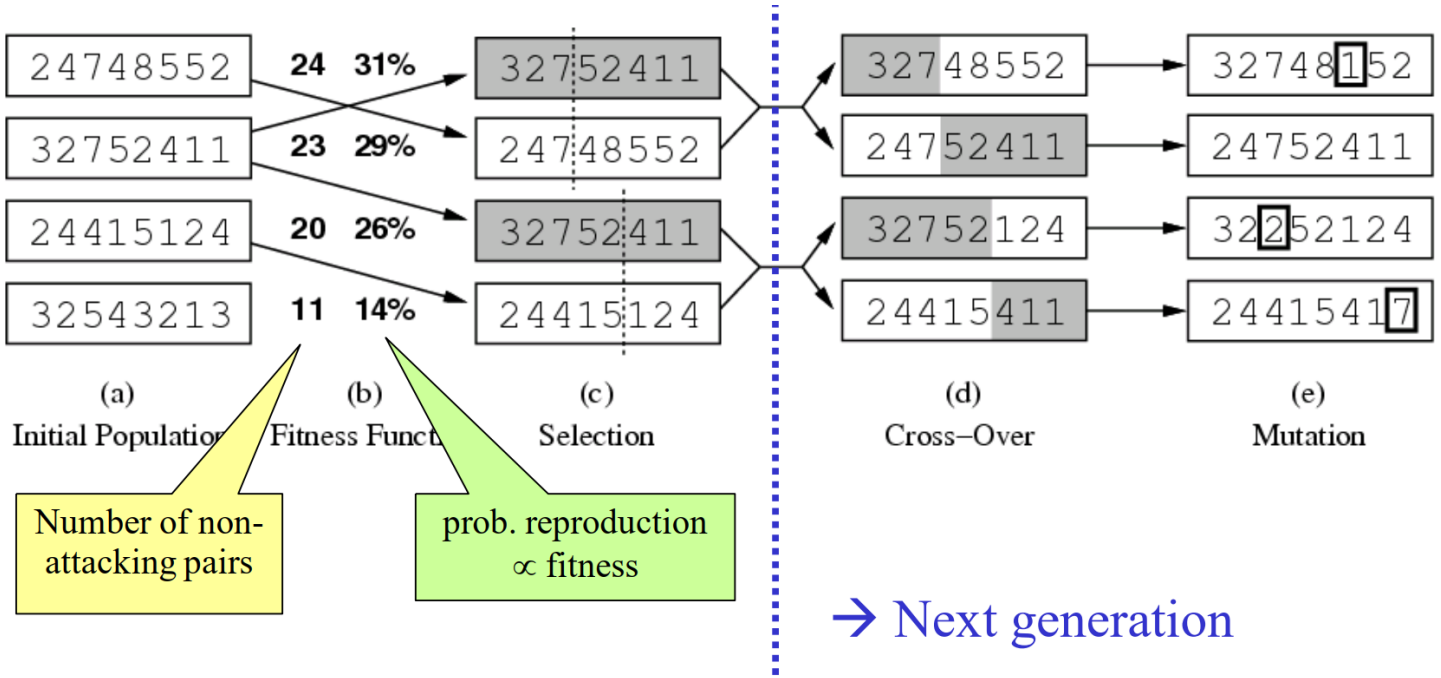
(3 2 7 5 2 4 1 1)

8-queens as 8 digits: digit i is the row of the queen in column i.

- The score $f(s)$ is called the **fitness** of s. Our goal is to find the global optimum (fittest) state
- At any time we keep a fixed number of states. They are called the **population**.

GENETIC ALGORITHM (GA) - 2

- **Genetic algorithm**: a special way to generate neighbors, using the analogy of cross-over, mutation, and natural selection.



One generation of 8-queens; fitness $f(s)$ = number of non-attacking pairs (28 at best, so we maximize). (a) population; (b) fitness and reproduction probability $f(s_i)/\sum_j f(s_j)$, e.g. $24/78 \approx 31\%$; (c) parents drawn with those probabilities; (d) one-point crossover at the dashed lines drawn in (c); (e) one digit mutated in three of the four children (boxed).

GA - One variation

1: Let $s_1, \ldots, s_N$ be the current population (N even)
2: Let $p_i = f(s_i) / \sum_j f(s_j)$ be the reproduction probability (needs $f \geq 0$)
3: **for** $k = 1;\ k < N;\ k+ = 2$ **do**
4: parent1 = randomly pick according to p_i
5: parent2 = randomly pick another, also according to p_i
6: randomly select a crossover point, swap strings of parents 1, 2 to generate children $t[k]$, $t[k+1]$
7: **end for**
8: **for** $k = 1;\ k \leq N;\ k+ = 1$ **do**
9: Randomly mutate each position in $t[k]$ with a small probability (mutation rate)
10: **end for**
11: The new generation replaces the old: $\{s\} \leftarrow \{t\}$.
12: Repeat

Every child must be visited by the mutation loop (line 8, step 1), although crossover (line 3) creates children in pairs.

Toronto Metropolitan University

PROPORTIONAL SELECTION

$$p_i = f(s_i) / \sum_j f(s_j)$$

$$\sum_j f(s_j) = 5 + 20 + 11 + 8 + 6 = 50$$

$$p_A = 5/50 = 10\%$$

Individual	Fitness	Prob.
A	5	10%
B	20	40%
C	11	22%
D	8	16%
E	6	12%

likewise $40\%, 22\%, 16\%, 12\%$ for B–E. A fit individual is only made *more likely* to reproduce, not certain to – which is what preserves diversity.

VARIATIONS OF GENETIC ALGORITHM

- Parents may survive into the next generation (*elitism*: keeps the best-so-far from being lost)
- Use ranking instead of $f(s)$ in computing the reproduction probabilities. This stops one very fit individual from taking over, and tolerates negative f.
- Cross over random bits instead of chunks (uniform instead of one-point crossover)
- Optimize over programs, usually represented as expression trees. Genetic programming.

GENETIC ALGORITHM ISSUES

- State encoding is the real ingenuity, not the decision to use a genetic algorithm. Crossover helps most when bits that are good together sit together in the string.
- Lack of diversity can lead to premature convergence and non-optimal solution
- Theory offers little practical guidance
  - Empirically crossover usually contributes more than mutation, but this depends on the encoding.
- Easy to implement.
- Try hill-climbing with random restarts first!